

Code Sentinel

System Design Document

1. Introduction

Code Sentinel is a multi-agent automated code review platform. Five independent specialist agents Security, Style, Performance, Logic and Documentation analyse a pull request or an on-demand diff in parallel. An orchestrator collects their findings, removes duplicates, ranks them by severity and then by confidence, attaches similar issues seen in the past, and returns one combined report.

The same report is delivered through three surfaces that must never disagree: a GitHub App (Check Run plus inline suggestions), a VS Code extension (side-panel review of the working tree), and a Next.js web dashboard (history, reports and configuration).

This document covers the system architecture, the database schema, the class, component, state and sequence diagrams, the REST API contracts, and the UI/UX wireframes.

1.1 Design principles

- Partial is a success state one slow or failing agent must not block the whole review.
- Nothing reaches a branch on its own an AI-suggested fix is committed only on an explicit developer action.
- Deterministic style fixes (ESLint / Prettier / Ruff / Black) involve no LLM and are safe to apply directly.
- Every read is scoped to one organization (tenant isolation).
- Agents are stateless and independently deployable behind one shared REST contract, so any one can be scaled, tested or reused on its own.

2. System Architecture

The system is organised into five layers. Clients reach only the API Gateway; nothing downstream is exposed to the internet.

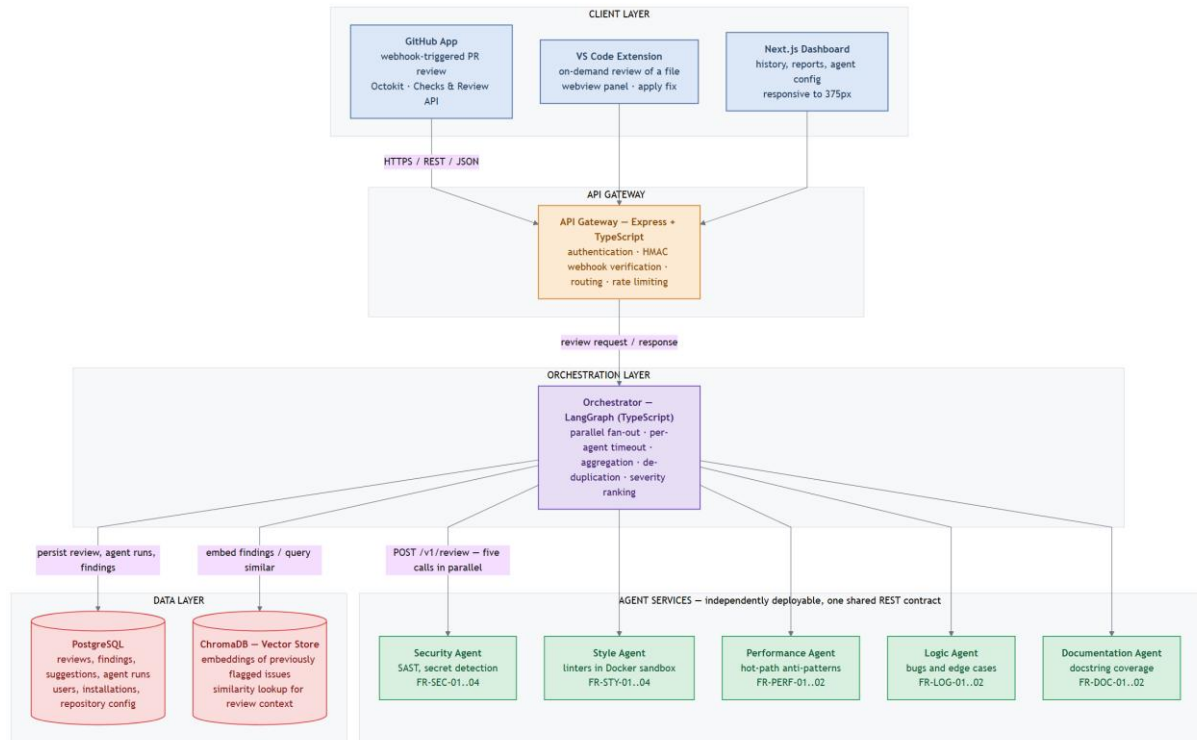


Figure 2.1 Layered architecture

- Client layer GitHub App (webhook-triggered, Octokit / Checks & Review API), VS Code extension (webview panel, apply-fix), Next.js dashboard (responsive to 375px).
- API Gateway Express + TypeScript. Authentication, HMAC-SHA256 webhook verification, routing, per-installation rate limiting.
- Orchestration layer LangGraph (TypeScript). Parallel fan-out, per-agent timeout, aggregation, de-duplication, severity ranking, vector-store context, Groq and Gemini quota budgets and large-diff planning.
- Agent services five independently deployable services, one shared REST contract (POST /v1/review).
- Data layer PostgreSQL (reviews, findings, suggestions, agent runs, users, installations, configuration) and ChromaDB (embeddings of previously flagged issues for similarity lookup).

2.1 Component view

Internal components of the gateway, orchestrator, a single agent service and the persistence layer.

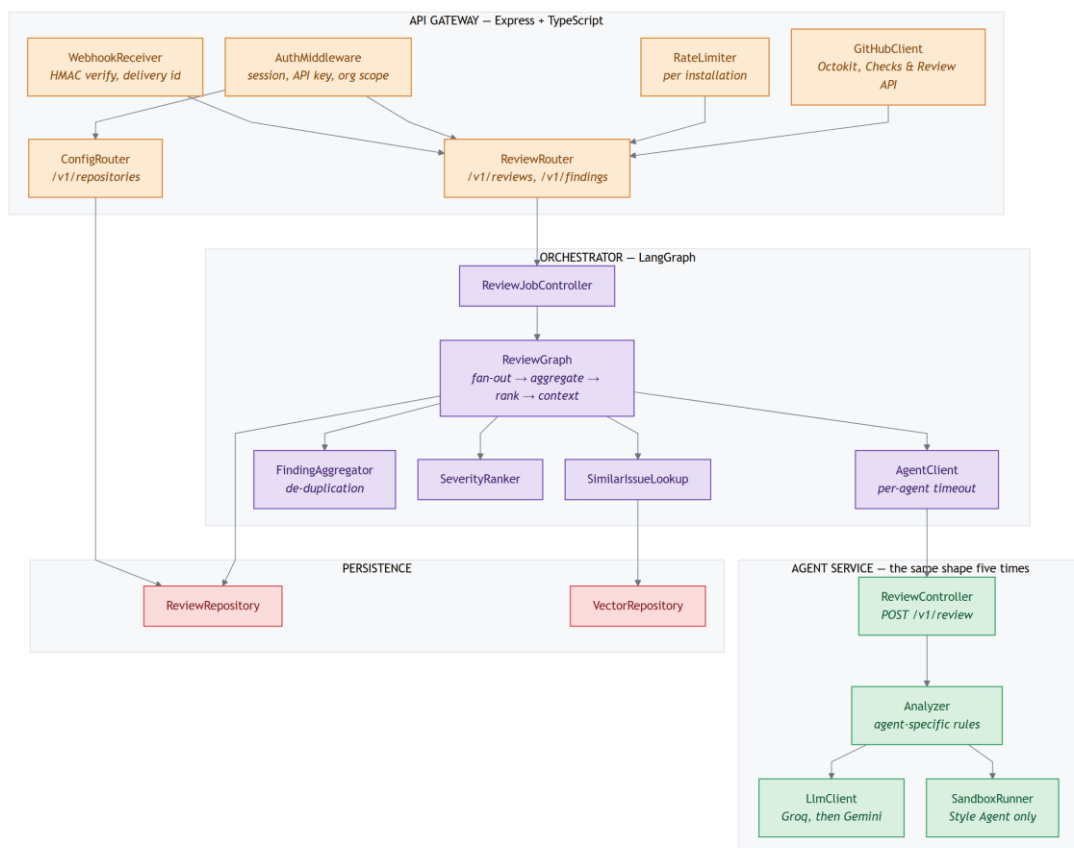
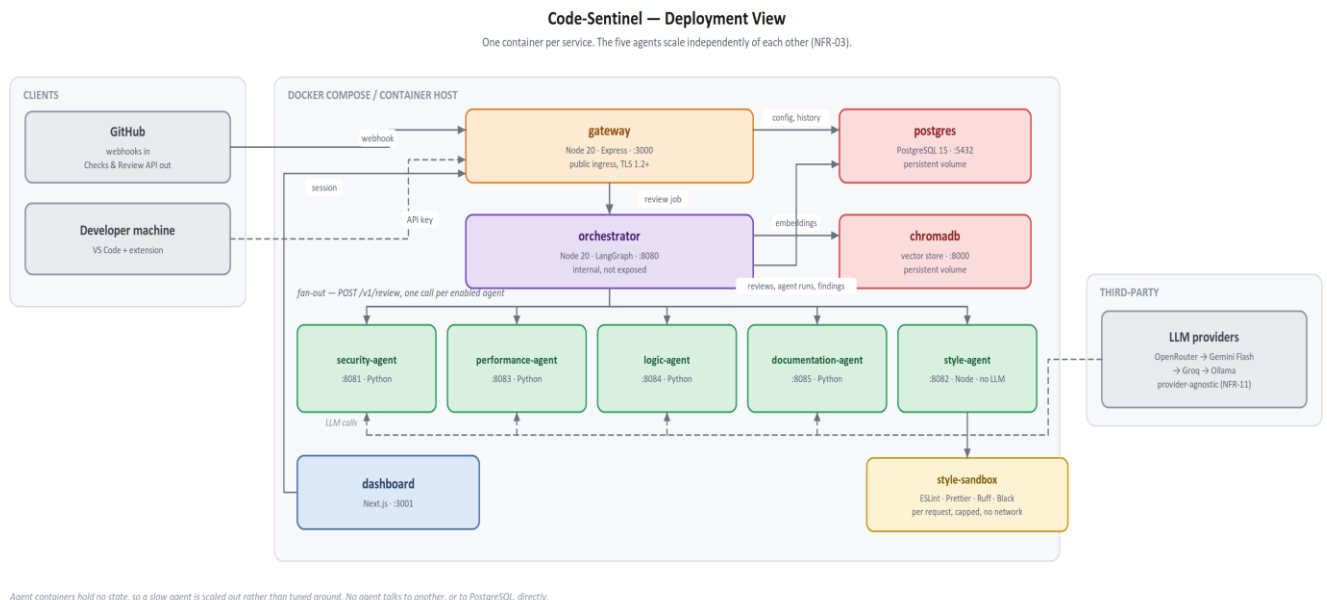


Figure 2.2 Component diagram

2.2 Deployment view

One container per service. The five agent containers hold no state, so a slow agent is scaled out rather than tuned around. No agent talks to another agent, or to PostgreSQL, directly.



Agent containers hold no state, so a slow agent is scaled out rather than tuned around. No agent talks to another, or to PostgreSQL, directly.

Figure 2.3 Deployment view

3. Database Schema (ER Diagram)

Target: PostgreSQL 15+. The schema falls into two logical groups identity / tenancy and the review pipeline. Vectors live in ChromaDB; PostgreSQL keeps the mapping (finding_embeddings, finding_similarities) so a finding can be traced both ways.

3.1 Enumerated types

Type	Values
org_role	admin, member
agent_kind	security, style, performance, logic, documentation
review_trigger	github_pull_request, vscode_on_demand, dashboard_manual
review_status	queued, running, completed, partial, failed
check_conclusion	pass, fail, neutral
agent_run_status	pending, running, succeeded, timed_out, failed, skipped
severity	critical, warning, info
fix_kind	deterministic, ai_suggested
suggestion_state	proposed, accepted, rejected, superseded

3.2 Identity and tenancy

- organizations / users / organization_members a GitHub organization, its users, and their admin or member role.
- github_installations the GitHub App installation. The installation token is never stored; it is minted per request from the App private key.
- sessions / api_keys dashboard session cookies and personal API keys for the VS Code extension; both store a hash only.
- repositories / repository_settings / repository_agent_config one repository, its thresholds and timeouts, and one enable flag per agent.

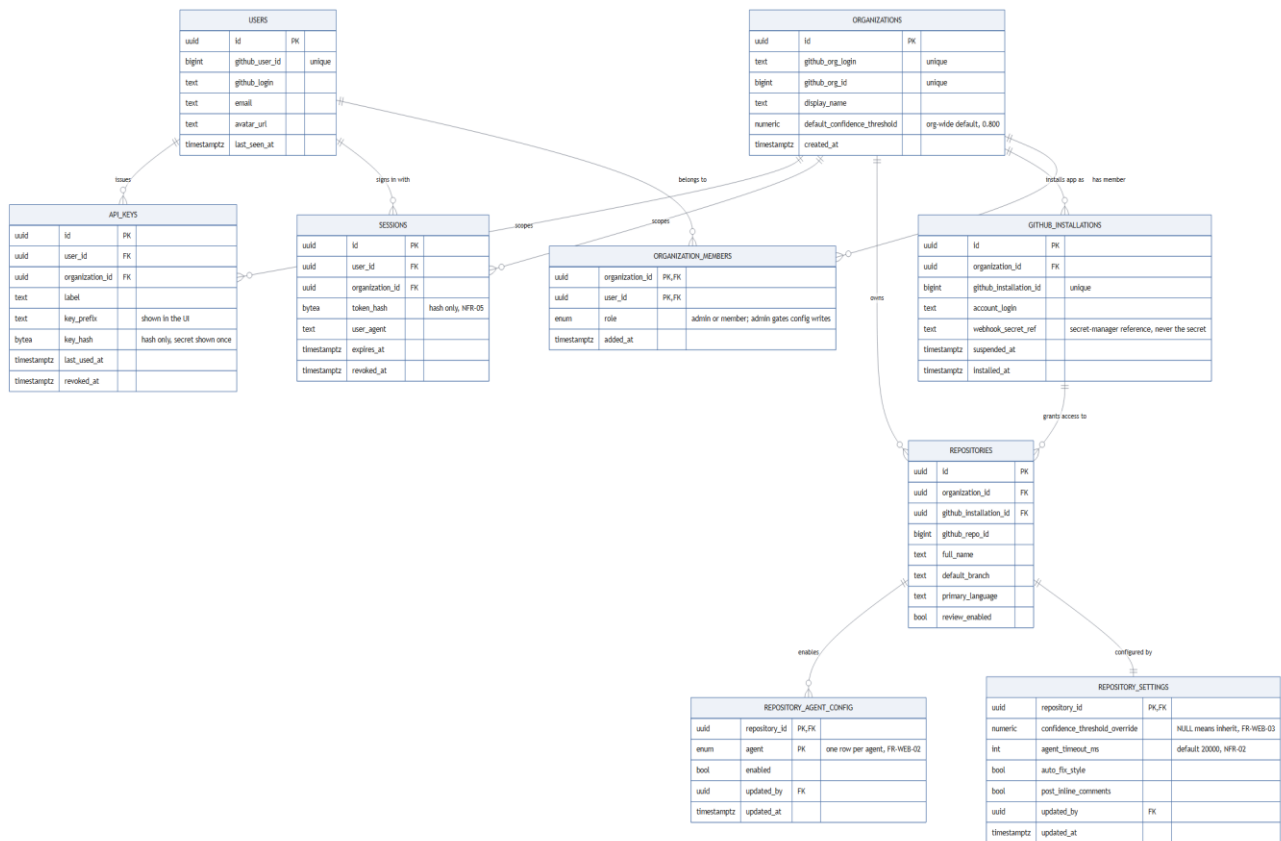


Figure 3.1 Identity, installations and per-repository configuration

3.3 Review pipeline

- **reviews** one review job, with trigger, commit SHAs, resolved confidence threshold, status and GitHub check-run linkage. A unique index on (repository, pr_number, head_sha) stops a redelivered webhook starting a duplicate job.
- **agent_runs** one row per agent invoked, written even on timeout, so per-agent latency, LLM provider, model and failure reason (including quota deferral) are always recorded.
- **findings** a single issue anchored to a file and line range, with severity, confidence and severity_rank. dedupe_key = hash(file_path, line range, rule/issue type) collapses the same issue reported by two agents.
- **suggestions** a proposed fix (deterministic or AI-suggested). An AI fix cannot be marked accepted without a resolving user.
- **finding_embeddings / finding_similarities** the ChromaDB collection / vector id for each finding and the similarity scores retrieved for review context.

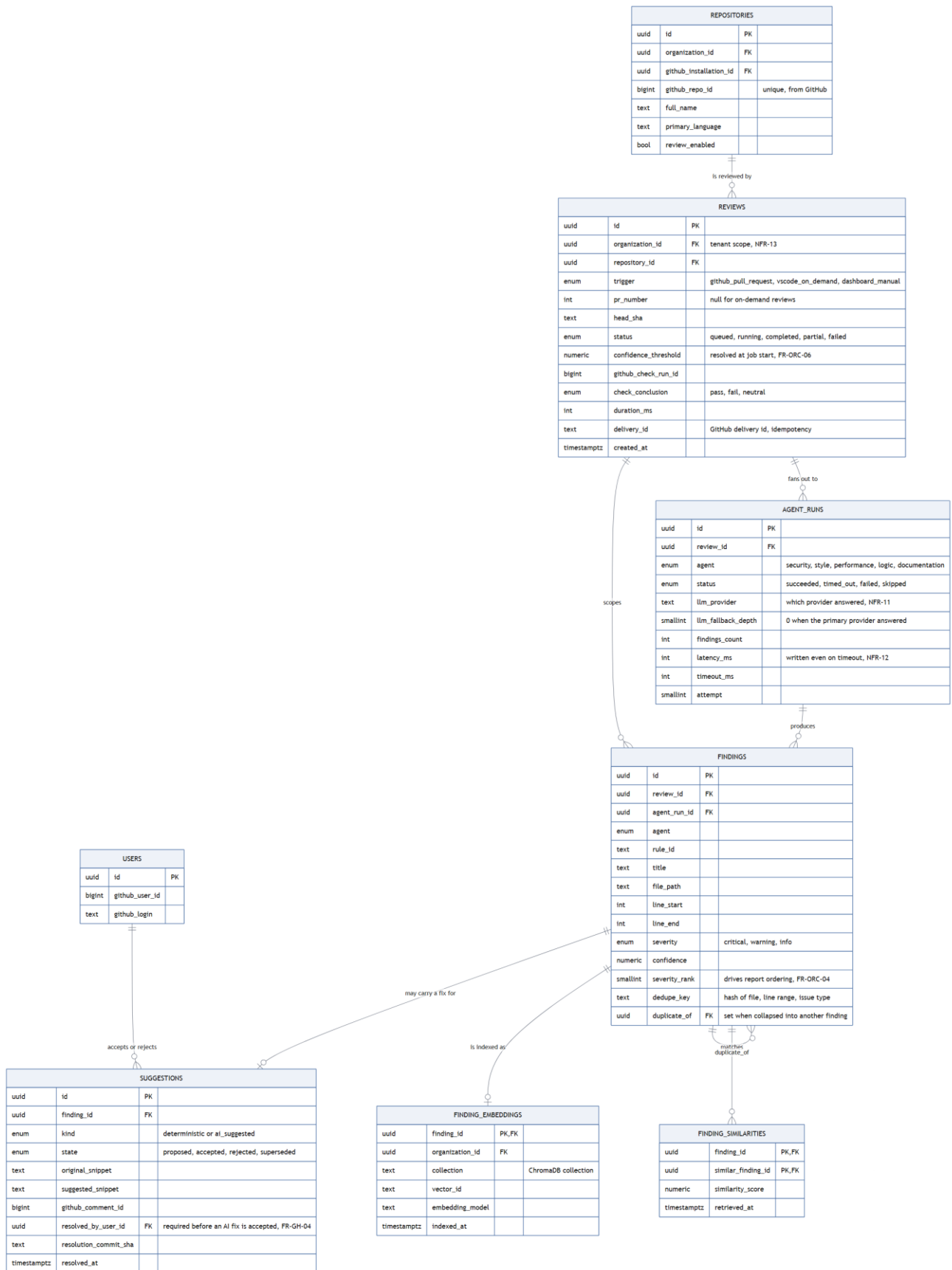
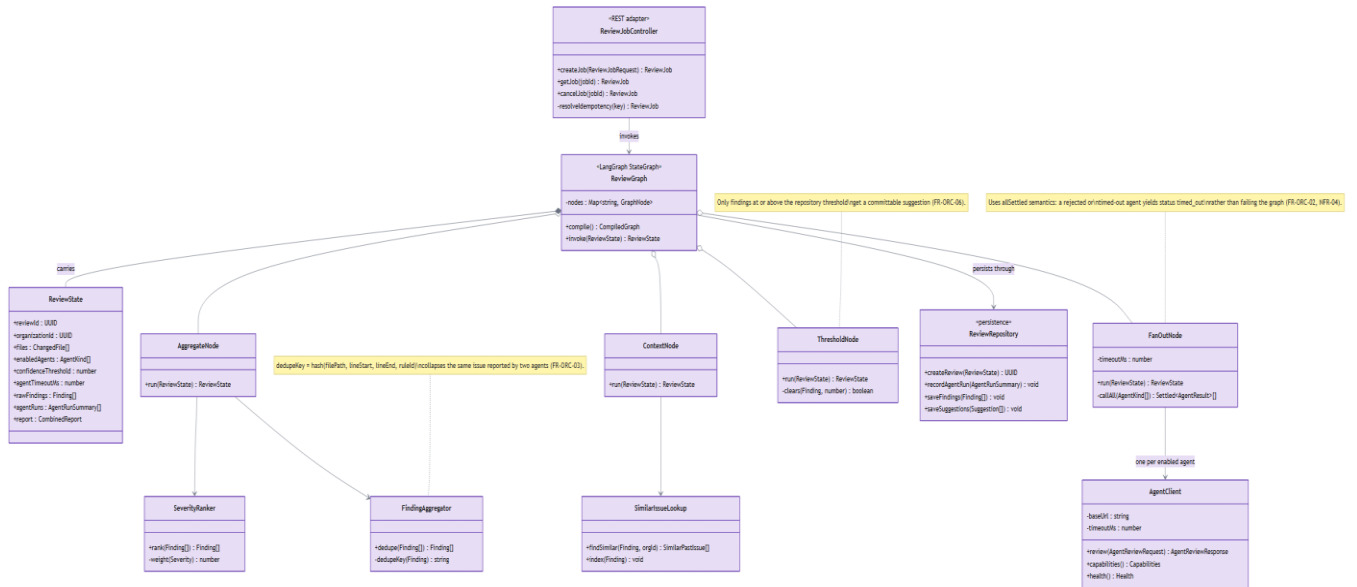


Figure 3.2 Reviews, agent runs, findings, suggestions and vector bookkeeping

4. Class, State and Sequence Diagrams

4.1 Orchestrator

The orchestrator is a LangGraph StateGraph. A ReviewState flows through fan-out, aggregate, context and threshold nodes and is persisted through a single repository. A rejected or timed-out agent yields status `timed_out` rather than failing the graph.



4.2 Agent services

All five services implement one ReviewAgent contract. BaseAgentService is abstract; each concrete agent supplies its own analyze(). The Style Agent is the only agent with no LlmClient its fixes are rule-based and run inside a network-disabled Docker sandbox.

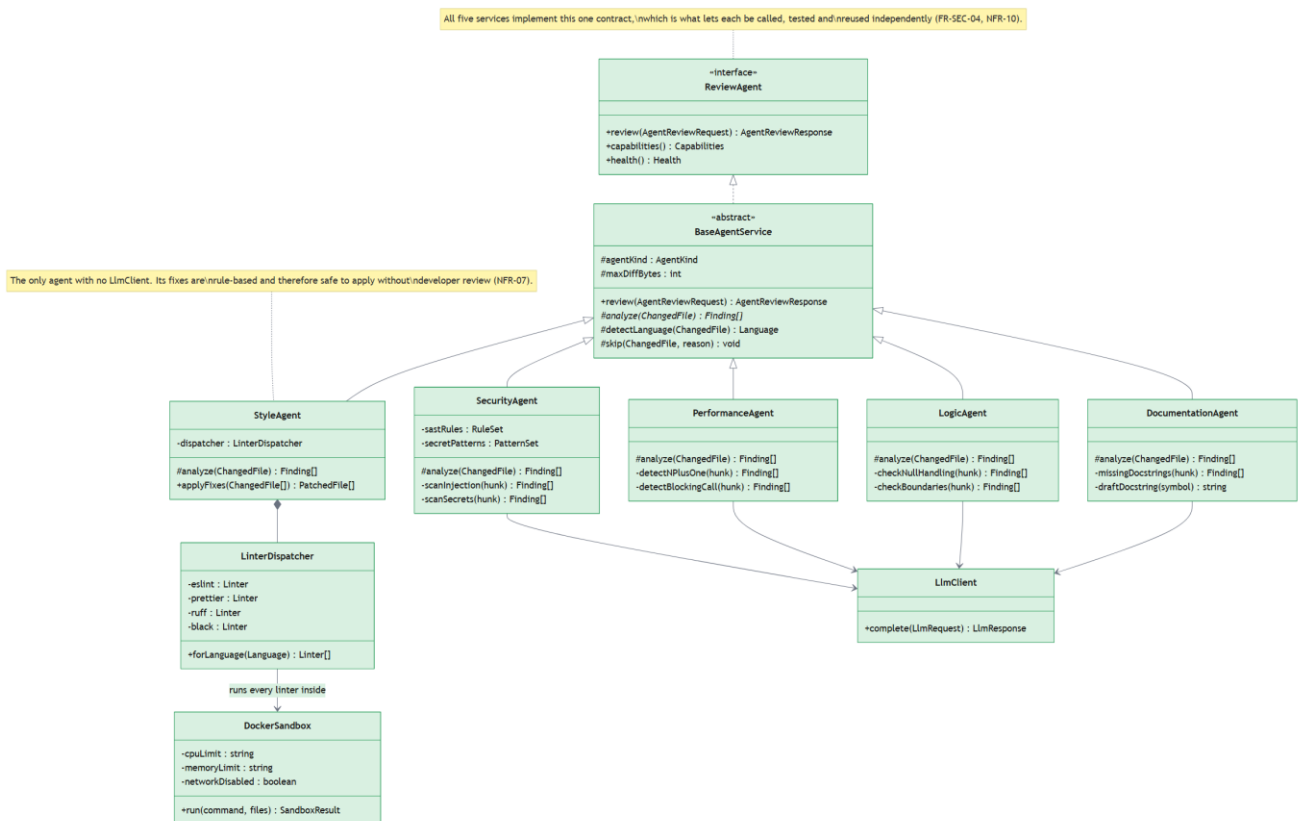


Figure 4.2 Agent service class diagram

4.3 LLM client, quota budget and large diffs

Groq is the primary LLM provider and Gemini the secondary. Groq runs openai/gpt-oss-120b and Gemini runs Gemini 3.5 Flash Lite; gemini-embedding-2 produces the 768-dimension vectors stored in ChromaDB, because Groq offers no embedding model. A batch goes to Groq when Groq's quota can take it now, otherwise to Gemini, and a Groq call that fails (rate limit, server error or timeout) is retried once on Gemini, recorded as fallback depth 1. Development and production use separate API keys, so local testing never consumes the production quota.

Production must never receive a rate-limit error, so the orchestrator reserves quota before sending each batch instead of retrying afterwards. One QuotaBudget per provider tracks requests and tokens over a sliding one-minute window and a rolling 24-hour window, and allows only 80% of each key's free-tier limits: Groq 30 RPM, 8K TPM, 1K RPD and 200K tokens per day; Gemini 15 RPM, 250K TPM and 500 RPD. Groq's 8K tokens per minute caps a Groq batch at about 3K diff tokens, so Groq takes roughly one batch a minute and Gemini takes the rest. A batch that fits neither provider waits in the queue; if it still cannot fit before its deadline, the LLM agent is marked skipped with `llm_quota_exhausted` and the review completes as partial. Reservations are settled with the token counts the answering provider reports.

Large diffs are skipped, never split. The gateway drops binary, generated and vendored files. For each LLM agent the orchestrator skips any file over 6K estimated tokens as `too_large`, marks files beyond 24K tokens per agent as `over_budget`, and packs the remaining whole files into batches of up to 12K tokens (about 3K on Groq), one LLM call per batch. A typical review costs four requests and about 60K tokens. The combined report lists every skipped file with its reason and a coverage summary, so a partial review states its gap rather than hiding it.

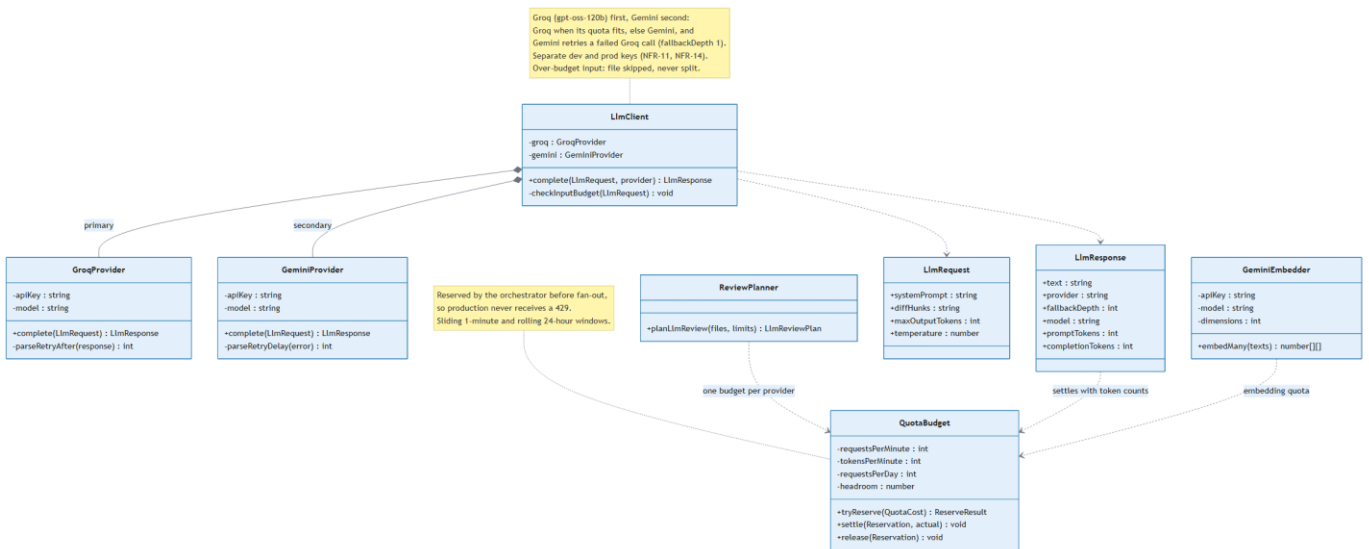


Figure 4.3 LLM client class diagram

4.4 Review job lifecycle

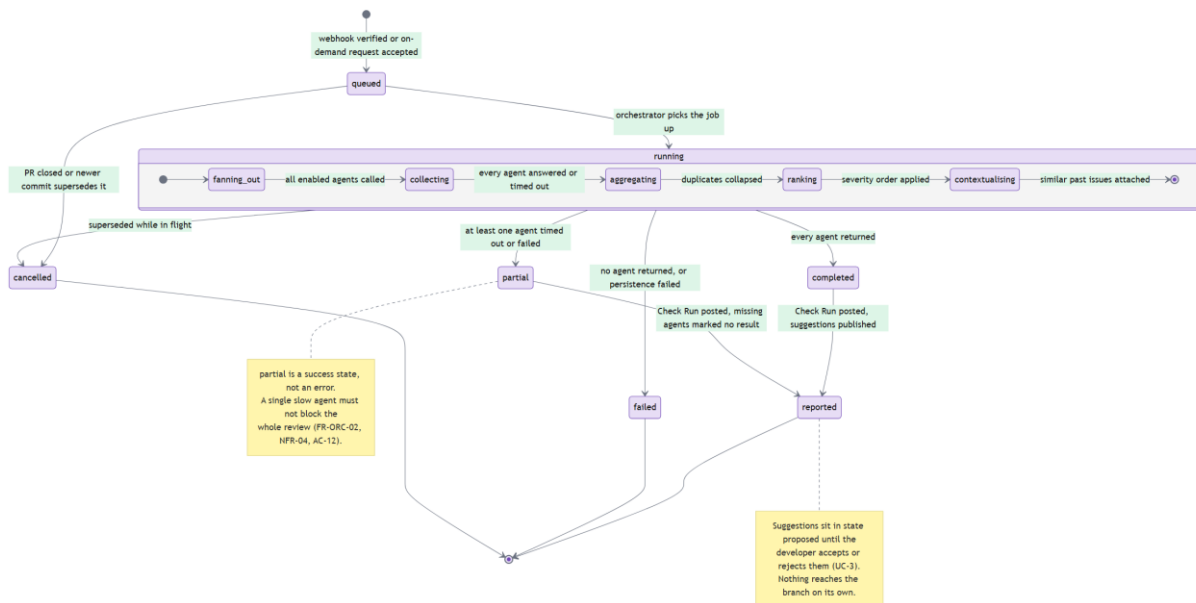


Figure 4.4 Review state machine. *partial* is a success state: a single slow agent must not block the whole review.

4.5 Sequence diagrams

UC-1 Automatic pull request review. A verified webhook triggers an asynchronous fan-out; a Check Run is created within five seconds; the timed-out agent is marked "no result" and the review still completes as partial.

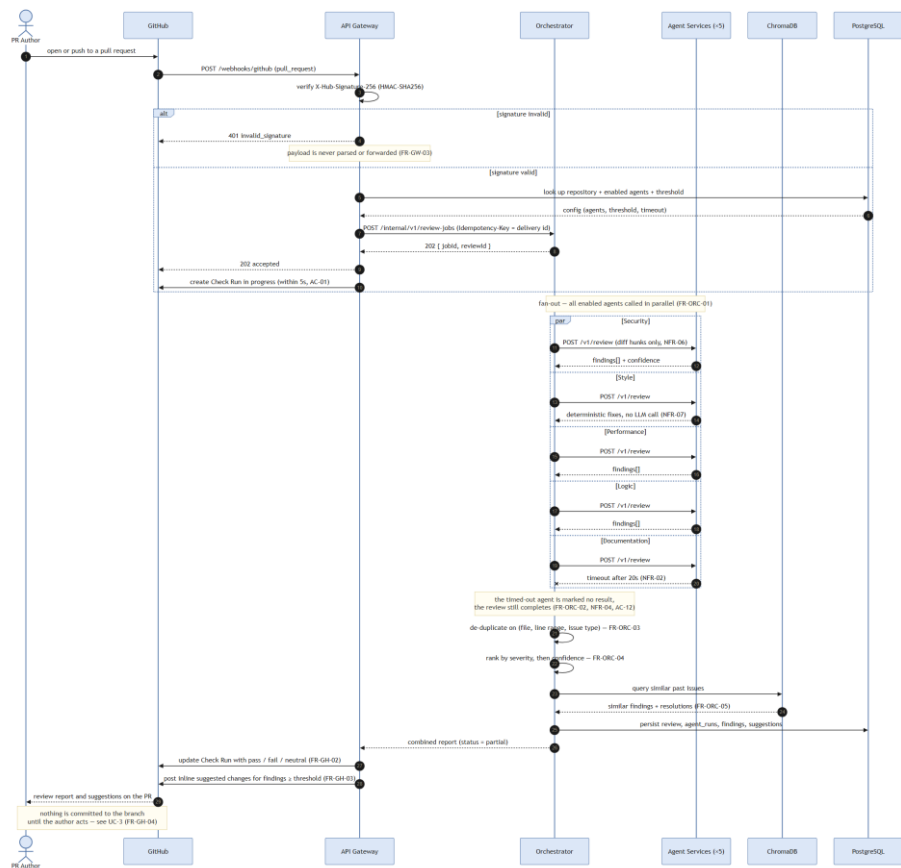


Figure 4.5 UC-1 automatic pull request review

UC-2 On-demand review from VS Code. The extension collects the active file and unsaved buffer, opens a webview panel, and fills in each agent section as server-sent events arrive.

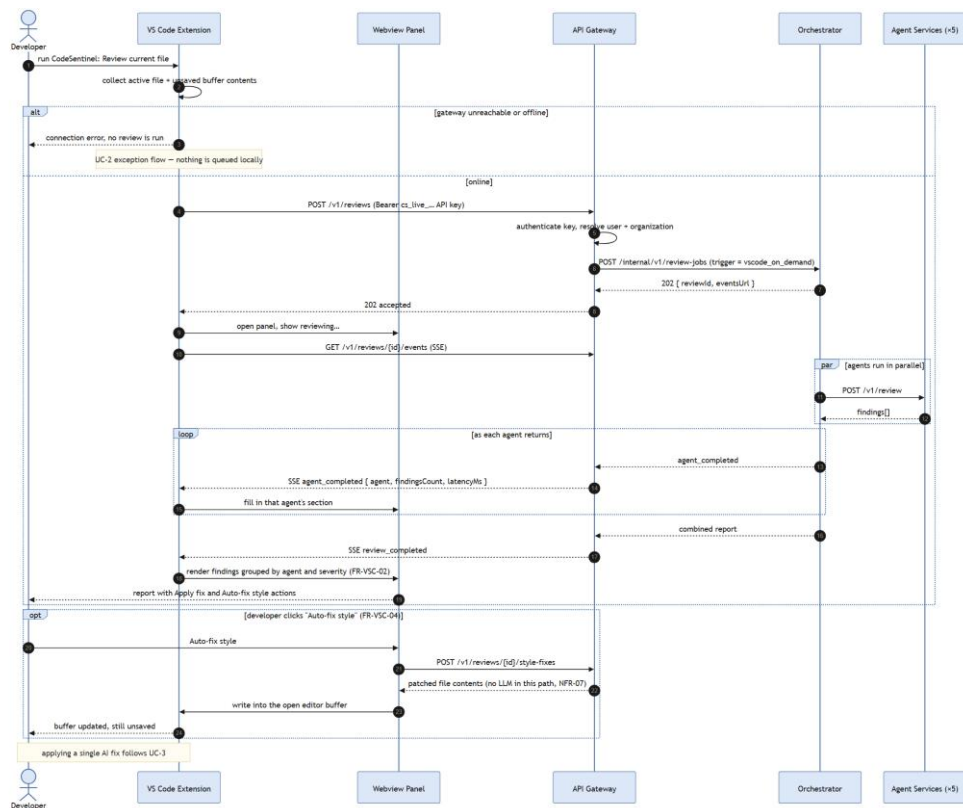


Figure 4.6 UC-2 on-demand review from VS Code

UC-3 Apply or reject a suggested fix. Acceptance is the only path to a branch commit; a superseded suggestion returns 409.

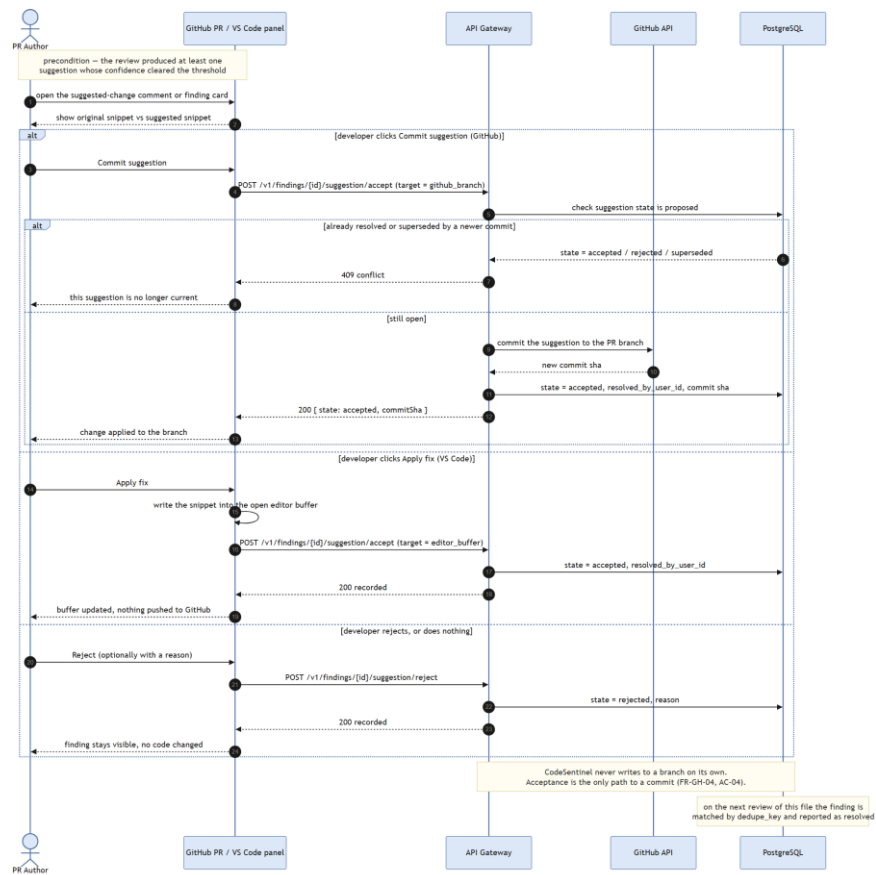


Figure 4.7 UC-3 apply or reject a suggested fix

UC-4 Configure repository and agent settings. Admin-only; disabling every agent is rejected; the change applies from the next review job, not retro-actively.

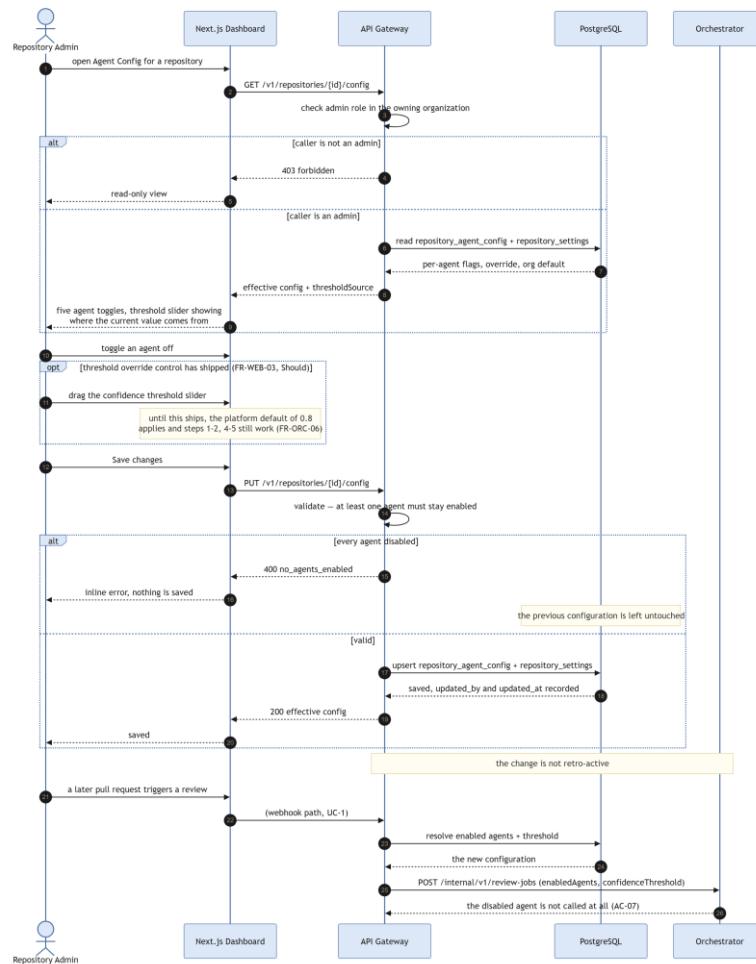


Figure 4.8 UC-4 configure repository and agent settings

UC-5 Browse review history. Every query is scoped to the caller's organization; no matching reviews produces an empty state rather than an empty table.

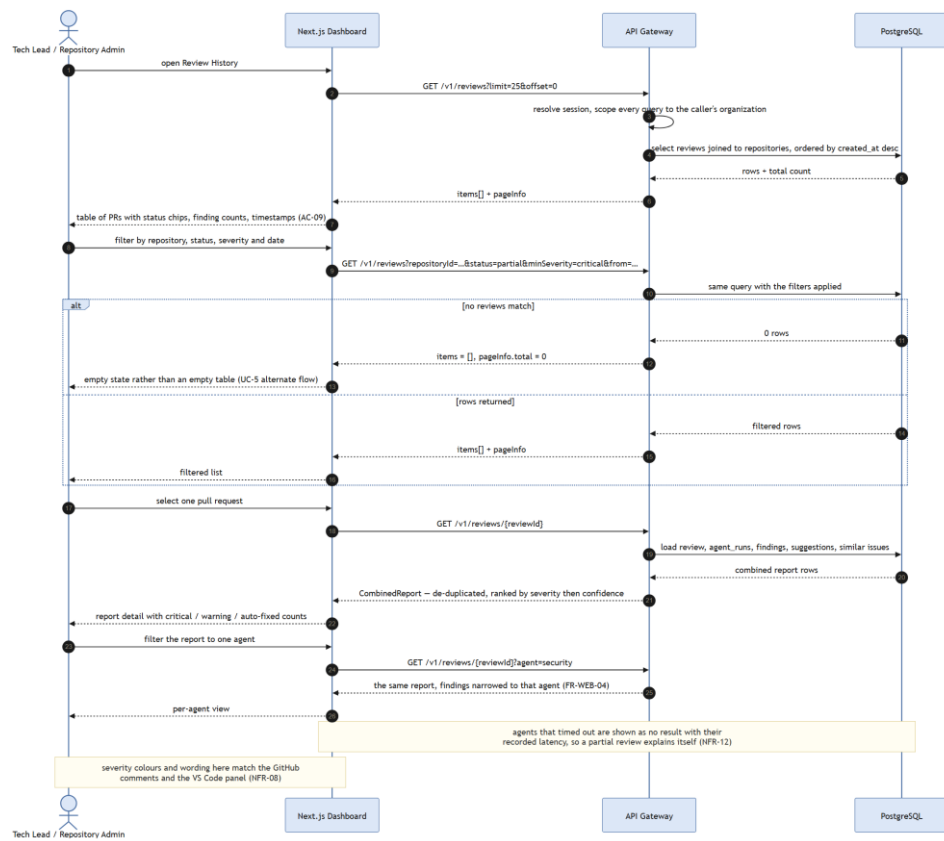


Figure 4.9 UC-5 browse review history

5. REST API Contracts (OpenAPI / Swagger)

Three OpenAPI 3.1 documents describe the system, plus a shared schema file (common.yaml) so that a Finding produced by an agent, carried by the orchestrator and rendered by a client is described exactly once.

- gateway.yaml the single public entry point for all three clients.
- orchestrator.yaml internal contract between the gateway and the LangGraph orchestrator; not exposed to the internet.
- agent.yaml one contract implemented five times by the agent services.
- common.yaml shared schemas: Finding, Suggestion, CombinedReport, AgentRunSummary, ChangedFile, SkippedFile, ReviewCoverage, Severity, Confidence, and the standard error responses.

5.1 API Gateway (gateway.yaml)

Base URL <https://api.codesentinel.dev>. Auth: GitHub webhooks by HMAC-SHA256 signature; dashboard by cs_session cookie (GitHub OAuth); VS Code by a personal bearer API key (cs_live_ prefix). Every non-webhook read is organization-scoped.

Method & path	Purpose	Use case
POST /webhooks/github	Receive a GitHub webhook event (signature verified before the body is read)	UC-1
POST /v1/reviews	Request an on-demand review	UC-2
GET /v1/reviews	List review history, filterable by repo, status, severity, trigger, date	UC-5
GET /v1/reviews/{reviewId}	Get the combined, de-duplicated, ranked report	UC-2, UC-5
GET /v1/reviews/{reviewId}/events	Stream review progress (SSE)	UC-2
POST /v1/reviews/{reviewId}/style-fixes	Apply all deterministic style fixes (no LLM in this path)	UC-2
POST /v1/findings/{findingId}/suggestion/accept	Accept a suggested fix; commits to the PR branch when target is github_branch	UC-3
POST /v1/findings/{findingId}/suggestion/reject	Reject a suggested fix, with an optional reason	UC-3
GET /v1/repositories	List repositories CodeSentinel is installed on	UC-4
GET /v1/repositories/{repositoryId}/config	Read effective agent configuration and where each value came from	UC-4
PUT /v1/repositories/{repositoryId}/config	Update agent configuration (admin only; at least one agent must stay enabled)	UC-4
GET /v1/me	Identify the caller and list organization memberships	Account
GET /v1/me/api-keys	List the caller's API keys (prefixes only)	Account
POST /v1/me/api-keys	Create an API key (plaintext secret returned once)	Account
DELETE /v1/me/api-keys/{keyId}	Revoke an API key	Account
GET /healthz	Liveness and readiness probe	Service

5.2 Orchestrator (orchestrator.yaml)

Base URL `http://orchestrator.internal:8080`. Auth: short-lived service bearer token. A review job is accepted immediately and runs asynchronously because a full fan-out can take up to a minute.

Method & path	Purpose
POST <code>/internal/v1/review-jobs</code>	Start a review and fan out to the enabled agents; Idempotency-Key (GitHub delivery id) makes a redelivery harmless
GET <code>/internal/v1/review-jobs/{jobId}</code>	Poll a job and read its combined report; agentRuns shows which agents have answered
POST <code>/internal/v1/review-jobs/{jobId}/cancel</code>	Cancel a running job when the PR is closed or superseded by a newer commit
GET <code>/internal/v1/agents/health</code>	Report the last known health of each agent service
GET <code>/healthz</code>	Liveness and readiness probe

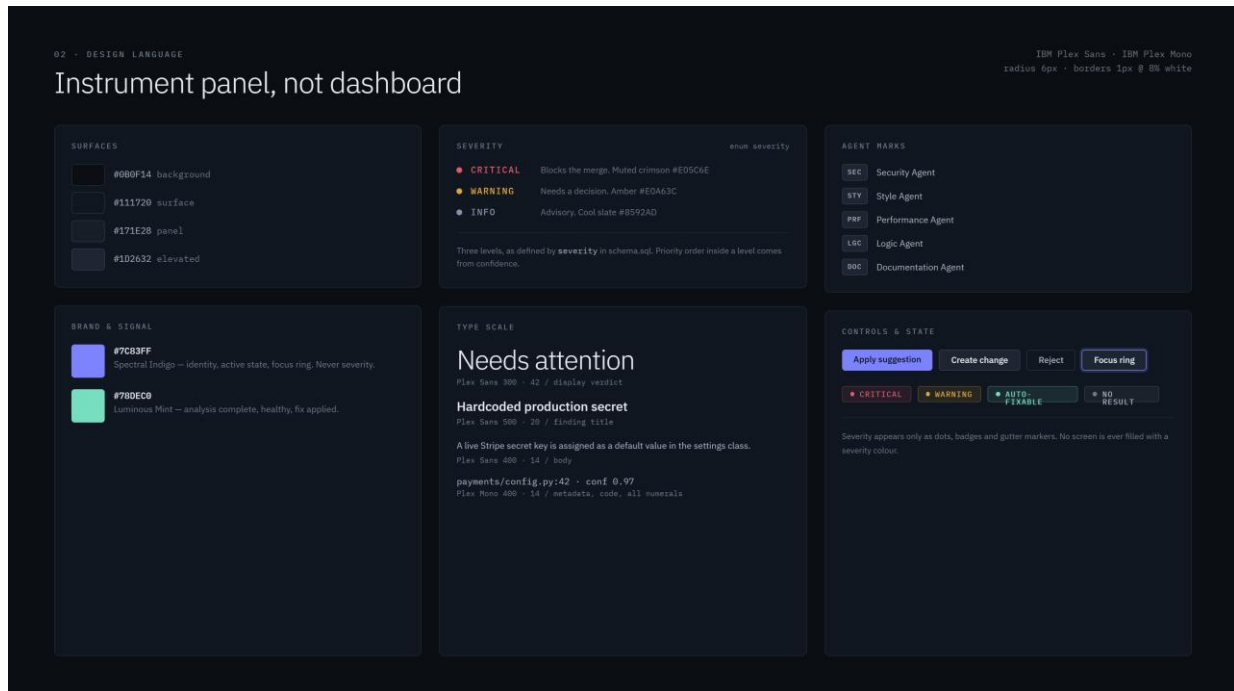
5.3 Agent service (agent.yaml)

One contract, implemented five times (ports 8081–8085). Auth: short-lived service bearer token. An agent receives a diff, returns findings, and is otherwise stateless; it never calls another agent and never writes to PostgreSQL.

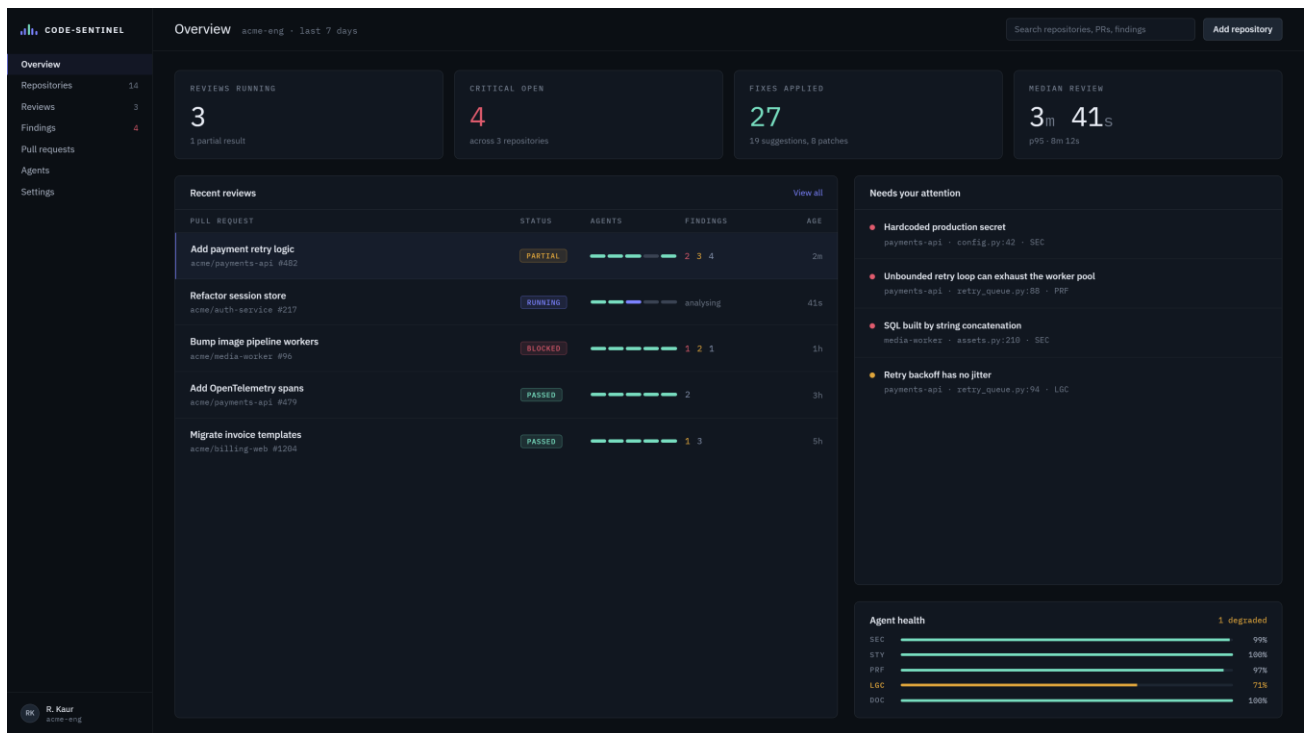
Method & path	Purpose
POST <code>/v1/review</code>	Analyze a diff and return findings with confidence; an empty findings array is a valid, successful result; files over the agent's size limit are returned as skipped, never split
GET <code>/v1/capabilities</code>	Describe the languages, rules and size limits (maxFileTokens) this agent supports; lets the orchestrator skip unsupported languages
GET <code>/healthz</code>	Liveness and readiness probe

6. Wireframes / UI-UX Mockups

15 frames on a 1920×1080 design canvas (exported at 2×). Every screen shares one scenario: pull request #482 "Add payment retry logic" on acme/payments-api. The run is partial the Logic agent exceeded its 90-second budget, the other four completed, and the report ships with the gap stated rather than hidden. Severity scale: critical / warning / info; ordering within a severity level comes from the agent's confidence score.



Frame 02 Design language: surfaces, brand, severity colours, type scale, agent marks, controls.



Frame 03 Dashboard: cross-repo view of what needs the user across 14 repositories, ordered by blocked merges.

ali

acme/payments-api / PR #482 · Add payment retry logic

Cancel reviewOpen on GitHub

ANALYSIS IN PROGRESS

Four of five agents have reported

Results appear as each agent finishes. You do not have to wait for the slowest one.

02:14
elapsed · budget: 05:00

SEC

Security

complete · 38s

110

STY

Style

complete · 22s

013

PRF

Performance

complete · 1m 04s

111

LGC

Logic

timed out · 02:00

Retrying once

DOC

Documentation

complete · 19s

002

The Logic agent exceeded its time budget and is being retried once.

If the retry also fails the review is published as a partial result, marked so nobody mistakes silence for a clean pass.

View partial results

Frame 04 Review in progress: four agents returned, Logic is over budget, findings are already readable.

acme/payments-spl / PR #482 · Add payment retry logic PARTIAL

rev_8f2a41 · 3m 42s · 12 files Re-run Open on GitHub

VERDICT · CHANGES REQUESTED

Two critical findings block this merge

A live payment secret is committed in `config.py`, and the new retry loop has no upper bound. Both have suggested fixes ready to apply.

SEC
Security
1 1 0

STY
Style
0 1 3

PRF
Performance
1 1 1

LGC
Logic
no result

DOC
Documentation
0 0 2

The Logic agent timed out twice and produced no result. Logic issues in this diff have not been checked.

Re-run Logic only

Findings 9 · ordered by severity, then confidence

ALL 9

CRITICAL 2

AUTO-FIXABLE 5

SEC	Hardcoded production secret in settings default	payments/config.py:42	0.97	FIX READY
PRF	Unbounded retry loop can exhaust the worker pool	payments/retry_queue.py:88	0.91	FIX READY
SEC	Webhook signature compared with == instead of a constant-time check	payments/webhooks.py:57	0.88	FIX READY
PRF	Retry backoff has no jitter — clients will synchronise	payments/retry_queue.py:94	0.84	FIX READY
STY	Broad except Exception swallows the failure cause	payments/retry_queue.py:103	0.79	
DOC	Public enqueue_retry() has no docstring	payments/retry_queue.py:71	0.72	FIX READY
STY	Magic number 1.5 should be a named constant	payments/config.py:41	0.68	FIX READY
DOC	Retry semantics are not documented in the module header	payments/retry_queue.py:1	0.64	
STY	Import block is not ordered per the repository convention	payments/webhooks.py:3	0.58	FIX READY

SUMMARY

The retry mechanism is sound in shape but unsafe in two places. The committed Stripe key must be revoked, not just removed from the diff — it is already in the branch history.

FILES TOUCHED

payments/retry_queue.py ●●●●●
payments/config.py ●
payments/webhooks.py ●
tests/test_retry_queue.py ●
+ 8 files with no findings

RUN DETAIL

review_id	rev_8f2a41
commit	c4e91b7
model	claude-sonnet-4
tokens	184,220
cost	\$0.61
cache hit	63%

Apply 5 suggested fixes

Export report

Frame 05 Review overview: the verdict, the five-agent matrix, and the priority queue. Answers "can I merge this".

acme/payments-api#482 / payments/retry_queue.py +64 -12

SplitUnifiedNext finding 1

CHANGED FILES12

payments/

retry_queue.py

config.py

webhooks.py

client.py

tests/

test_retry_queue.py

test_webhooks.py

infra/

worker.tf

6 files clean

1 file skipped - generated

84

85

86

87

88 +

89 +

90 +

91 +

92 +

93 +

94 +

95

96

97 -

98

99

100

101

102

103

104

105

106

107 +

108 +

109 +

110 +

111 +

112 +

113

114

115

async def process_retries(self, batch_id: str) -> None:

pending = await self.store.claim(batch_id)

log.info("claimed %d retries", len(pending))

while True:

failed = await self._dispatch(pending)

if not failed:

break

pending = failed

await asyncio.sleep(self.backoff * attempt)

await self.store.release(batch_id)

log.debug("done")

log.info("batch %s complete", batch_id)

except Exception:

log.warning("retry failed")

await self.store.release(batch_id)

async def _dispatch(self, jobs: list[Job]) -> list[Job]:

results = await asyncio.gather(

*(self.client.send(j) for j in jobs),

return_exceptions=True,

)

return [j for j, r in zip(jobs, results) if isinstance(r, Exception)]

def enqueue_retry(self, job: Job) -> None:

self.store.push(job, delay=self.settings.retry_delay(job.attempt))

FINDINGS IN THIS FILE4

CRITICAL

Unbounded retry loop can exhaust the worker pool

PBP

L88

WARNING

Retry backoff has no jitter — clients will synchronise

PBP

L94

WARNING

Broad except Exception swallows the failure cause

STY

L103

INFO

Public enqueue_retry() has no docstring

DOC

L71

The Logic agent produced no result for this review, so control-flow issues in this file are unchecked.

Apply 2 fixes in this file

Frame 06 Three-panel code review: files / diff / findings, with findings anchored to the line that produced them.

Back to review

find_2c71w9

CRITICAL

PERFORMANCE AGENT

payments/retry_queue.py : 88-92

Unbounded retry loop can exhaust the worker pool

WHAT THE AGENT SAW

WHY IT MATTERS HERE

The loop at line 88 exits only when `_dispatch` returns an empty list. A payment that fails deterministically — expired, blocked, wrong currency — is returned as failed on every pass, so the loop runs until the process is killed.

This coroutine holds a worker slot for its whole lifetime. The pool is 16 wide. Sixteen permanently failing payments take the entire retry service down, and the batch is never released.

Suggested fix

TESTS PASS

+4 -2 · uses existing max_attempts

```
88 - while True:
89 + for attempt in range(1, self.max_attempts + 1):
90     failed = await self._dispatch(pending)
91     if not failed:
92         break
93     pending = failed
94 +
95 + await self.store.dead_letter(pending)
96 - await asyncio.sleep(self.backoff * attempt)
97 + await asyncio.sleep(self.backoff * attempt)
```

Apply as suggestion

Create a branch

Reject...

Applying posts a GitHub suggestion on line 88

CONFIDENCE

0.91

High. The exit condition is directly readable in the diff and `max_attempts` already exists in config.

EVIDENCE

Loop has no counter and no timeout
retry_queue.py:88

`max_attempts = 5` defined but never read
config.py:18

Worker pool capped at 16 concurrent tasks
worker.tf:31

RULE

PRF-014 · unbounded-retry

Enabled at organisation level. Triggered 4 times in the last 90 days, rejected once.

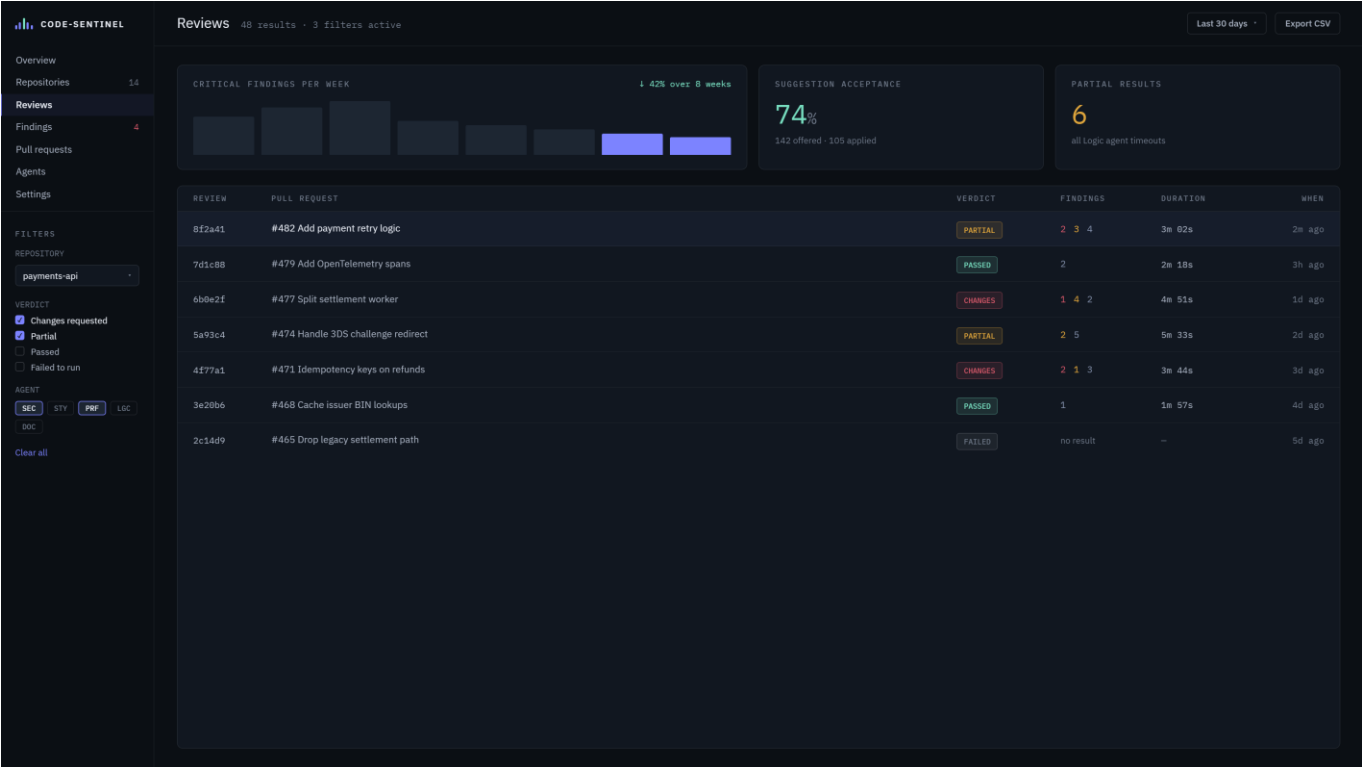
DISCUSSION

Agreed — dead-letting after 5 is what the runbook already says we do.

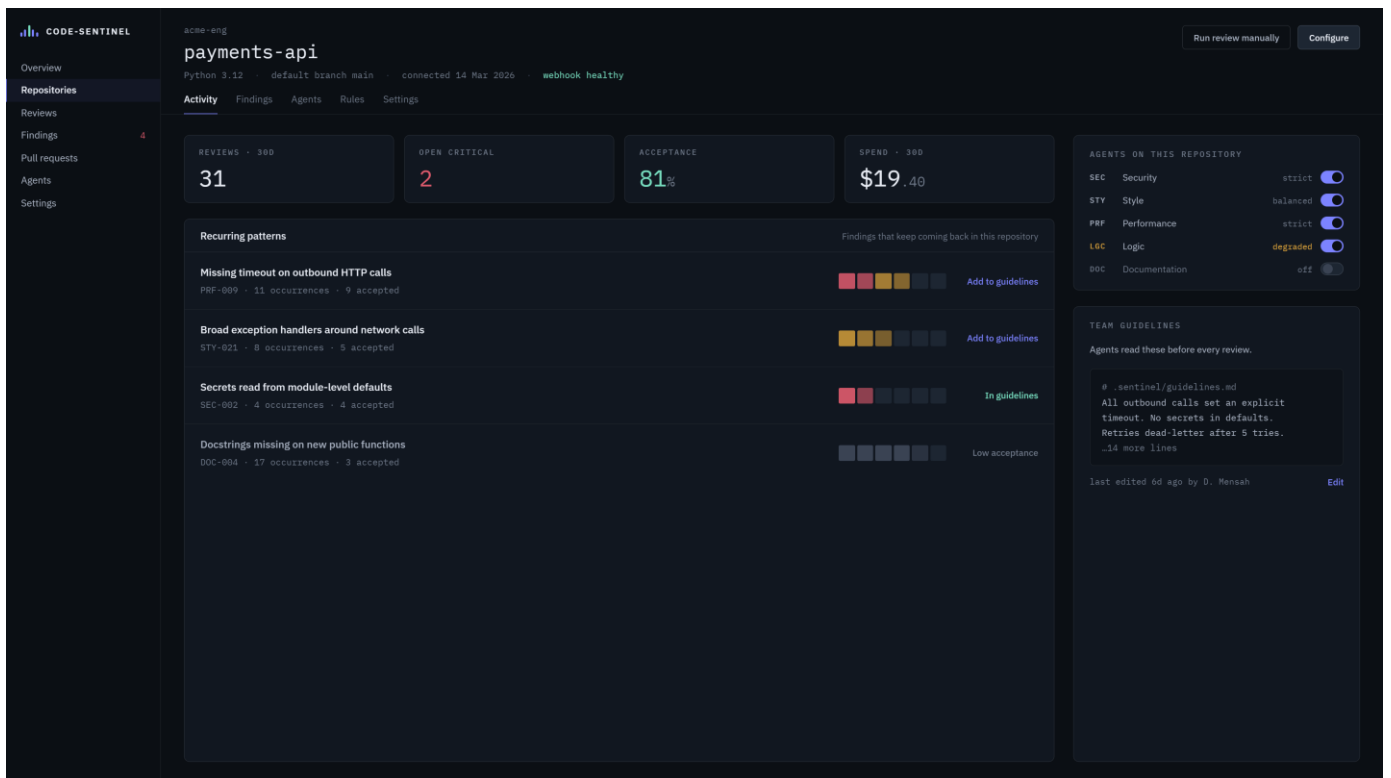
D. Mensah · 4m ago

Reply to this finding

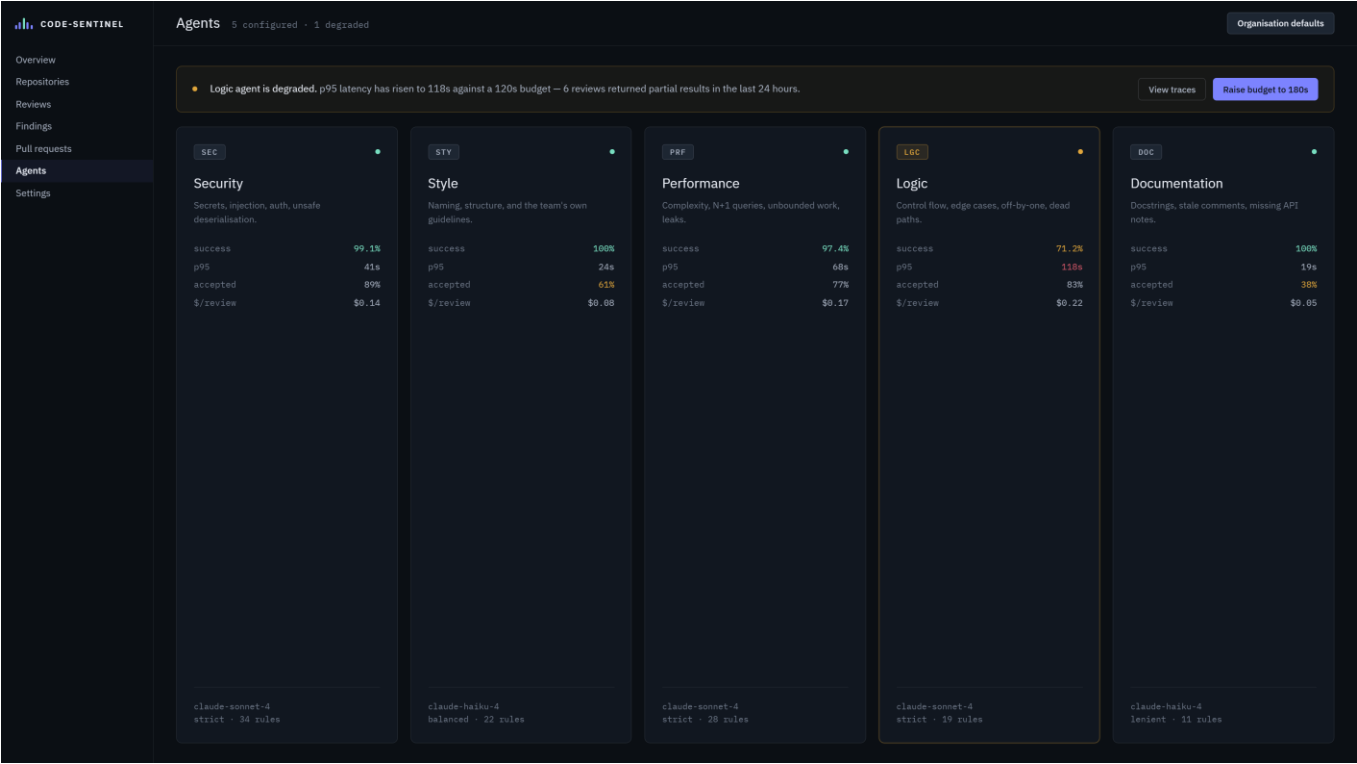
Frame 07 Finding detail & fix: evidence, reasoning, proposed patch, confidence, rule provenance. Apply, branch, or reject with a reason.



Frame 08 Review history: filterable record of past reviews. Answers "is the codebase getting better".



Frame 09 Repository detail: active agents, recurring problems, and where the repository's rules came from.



Frame 10 Agents as operational objects: success rate, p95 latency, acceptance rate, cost per review.

CODE-SENTINEL

Overview

Repositories

Reviews

Findings

Agents

Settings

acee / payments-api / Configuration

unsaved changes

Discard

Save configuration

Agents

Triggers

Ignore paths

Budget & limits

Notifications

Access

Agent behaviour on this repository

Blocking agents can fail the merge check. Advisory agents post findings without failing it. Changes apply to the next review.

AGENT	SCOPE	MODE	MIN. SEVERITY	MODEL
☒ Security	**/*.py, **/*.ts, infra/**	BLOCKING ADVISORY	warning	sonnet-4
☒ Style	**/*.py	BLOCKING ADVISORY	info	haiku-4
☒ Performance	**/*.py, !tests/**	BLOCKING ADVISORY	warning	sonnet-4
☒ Logic timed out on last 3 reviews	**/*.py, !migrations/**	BLOCKING ADVISORY	warning	sonnet-4
☐ Documentation	disabled on this repository	—	—	—

TRIGGERS

☒ Pull request opened

☒ New commits pushed to an open PR

☐ Merge into main

☒ Manual re-run from dashboard or CLI

IGNORE PATHS

migrations/**

vendor/**

**/*.pb2.py

tests/fixtures/**

Matched files are never sent to an agent. 1,204 files excluded on the last review.

BUDGET & LIMITS

Per-agent timeout

90s

Max cost per review

\$1.50

Publish partial results when an agent fails

☒

acee-engineering

m.okafor · admin

Frame 11 Repository & agent configuration: per-agent mode, scope, thresholds, triggers, ignore paths, budgets.

The merge check and the inline comment

Open Add payment retry logic #482

jrivera wants to merge 6 commits into main from feat/retry-queue

Some checks were not successful

build / test Successful in 2m 14s Details

code-sentinel / review PARTIAL 2 critical, 2 warnings across 4 files. Logic agent did not complete — its findings are missing from this run. Details

coverage 88.4% · no decrease Details

Merging is blocked

Two blocking findings — one from Security, one from Performance — must be resolved or explicitly dismissed with a reason.

Code-Sentinel reviewed 4 minutes ago rev_8f2a41

The retry mechanism is sound in shape but unsafe in two places. The committed Stripe key must be revoked, not just removed from the diff — it is already in the branch history.

2 critical 2 warning 5 info 5 suggestions ready to commit

Merge pull request

Re-run Code-Sentinel

payments/config.py +31 -4

```
39 timeout_seconds: int = 30
40 max_retries: int = 5
41 + backoff_base: float = 1.5
42 + stripe_secret: str = "sk_live_51H8vQ2Lm9pRtVwYz"
43 webhook_url: str = env("PAYMENTS_WEBHOOK")
```

Code-Sentinel CRITICAL · SECURITY conf 8.97

Hardcoded production secret

A live Stripe secret key is committed as a default value. Anyone with repository read access can use it against production. Load it from the environment and rotate the exposed key.

SUGGESTED CHANGE

```
- stripe_secret: str = "sk_live_51H8vQ2Lm9pRtVwYz"
+ stripe_secret: str = env("STRIPE_SECRET_KEY")
```

Commit suggestion

Dismiss with reason

CWE-798

payments/retry_queue.py +64 -11

```
retry_queue.py:88
```

Code-Sentinel CRITICAL · PERFORMANCE conf 9.91

Unbounded retry loop can exhaust the worker pool

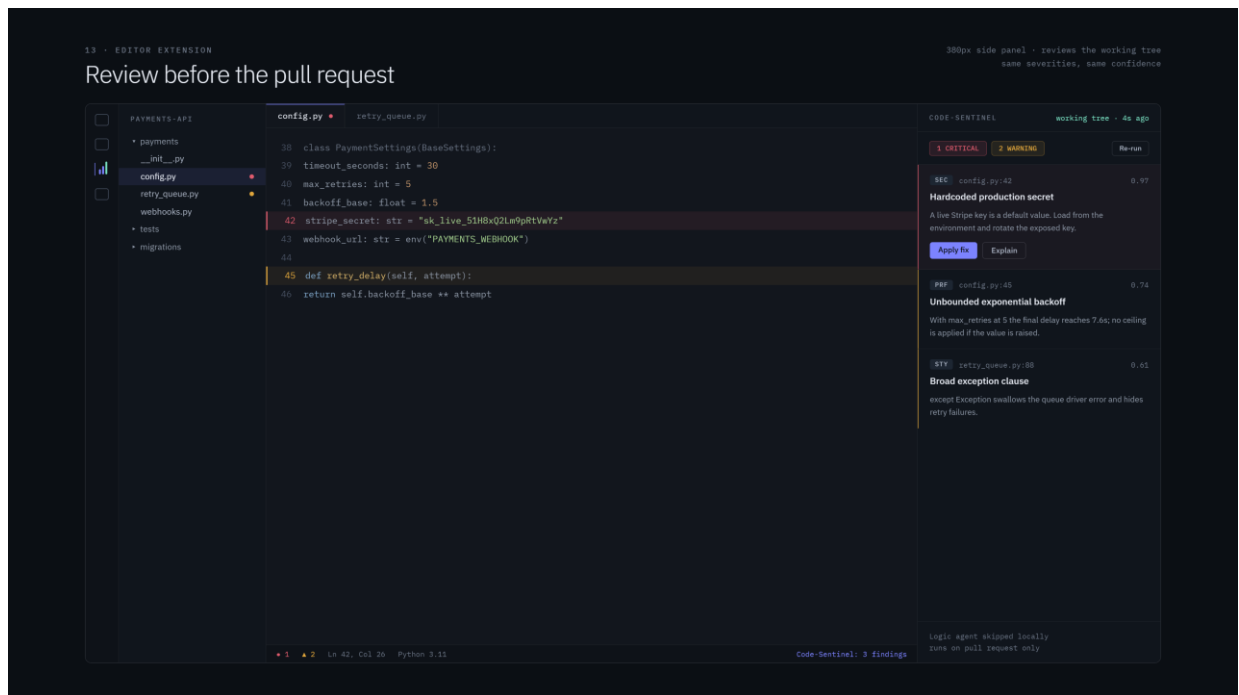
Each failed job is re-enqueued without a ceiling on total attempts. A permanently failing payment will occupy a worker indefinitely. Cap attempts at max_retries and move exhausted jobs to a dead-letter queue.

Commit suggestion

Dismiss with reason

retry_queue.py:88

Frame 12 Pull request check: the merge check and two inline comments, where most developers meet the product.



Frame 13 Editor extension: the same review in a 380px side panel, run against the working tree.

Every state names its cause and one next action

FIRST RUN · NO REPOSITORIES



No repositories connected

Install the Code-Sentinel app on an organisation and choose which repositories it may read. Reviews start on the next pull request.

[Connect a repository](#)[Read the setup guide](#)

CLEAN PASS · NO FINDINGS

[SEC](#) [STY](#) [PDR](#) [LOC](#) [DOC](#)

Nothing to report

All five agents completed against 7 files and 214 changed lines. No finding met the reporting threshold for this repository.

42s \$0.71 [merge check passed](#)[View what was checked →](#)

DEGRADED · AGENT UNAVAILABLE

Logic agent did not complete

The agent exceeded its 90-second budget on `retry_queue.py`. The other four agents finished and their findings are published. This review is marked partial.

```
agent_timeout · review #12345 · attempt 2 of 2
budget 90s · elapsed 98s · 1.842 lines in scope
```

[Re-run Logic agent only](#)[Accept partial result](#)

FAILED · REVIEW COULD NOT START

Repository access was revoked

Code-Sentinel can no longer read `acme/payments-api`. The installation token was rejected, so no agent was dispatched and nothing was billed.

```
403 Installation revoked · queued 3 reviews
last successful review · 6 Feb, 14:02
```

[Reauthorise installation](#)[Contact an org admin](#)

Frame 14 Empty & error states: first run, clean pass, degraded run, revoked access. Each names its cause and offers one next action.